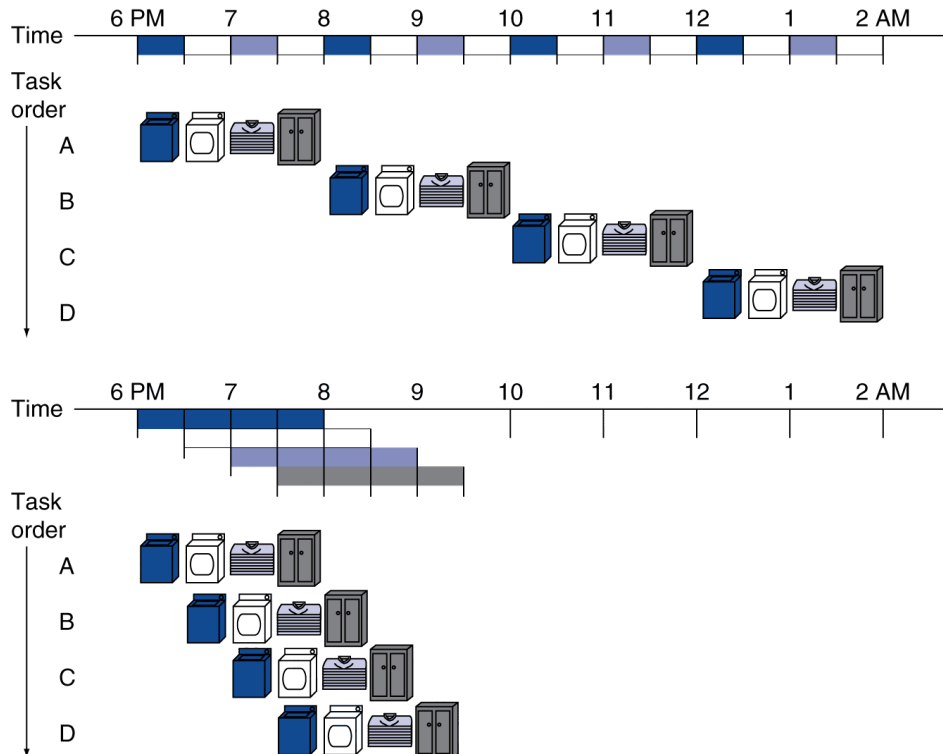


# Pipelining Analogy

- Pipelined laundry: overlapping execution
- Parallelism improves performance



■ Four loads:

■ Speedup  
 $= 16/7 = 2.3$

■ Non-stop:

■ Speedup  
 $= 2n/0.5n + 1.5 \approx 4$   
 $= \text{number of stages}$

# Pipeline

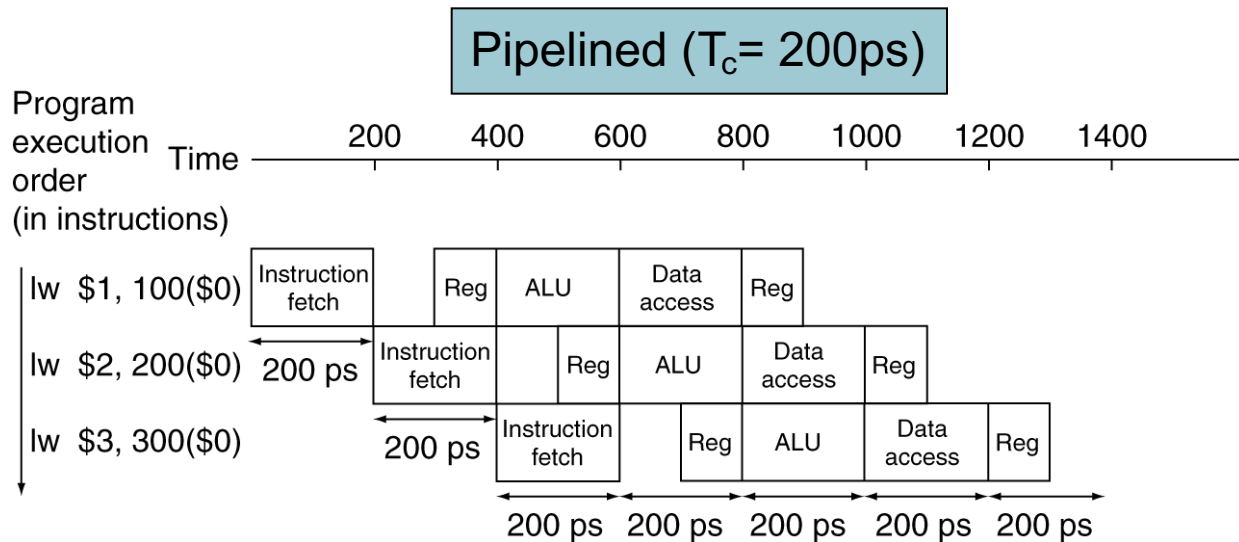
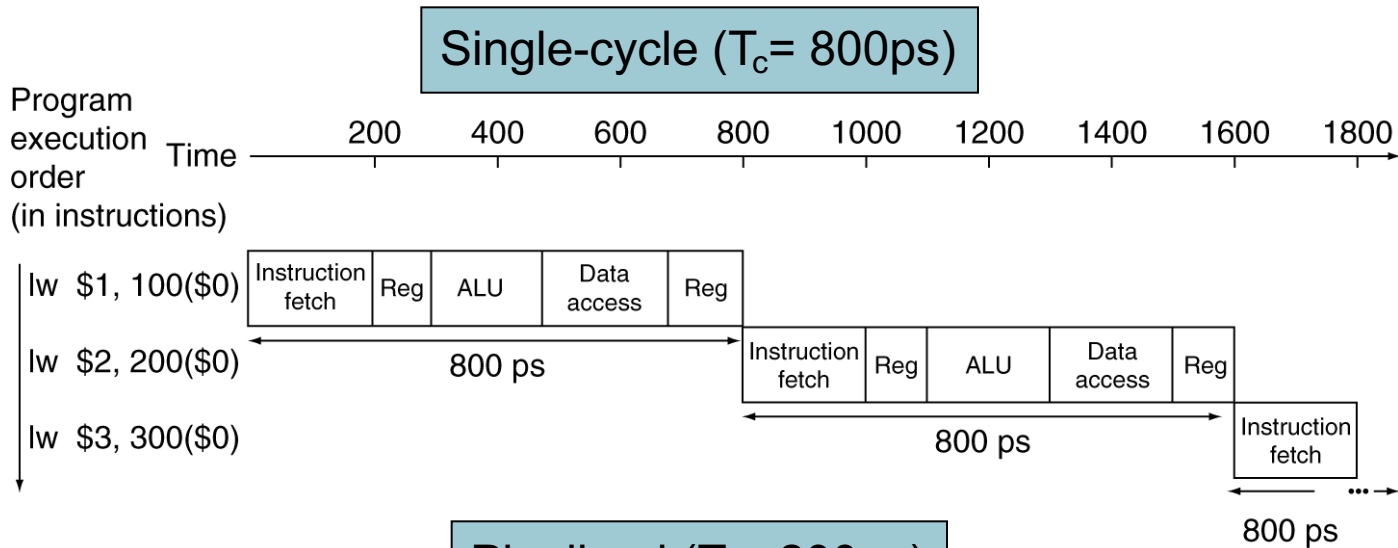
- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

# Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath.
- What is the time per instruction?

| Instr       | Instr fetch | Register read | ALU op | Memory access | Register write | Total time         |
|-------------|-------------|---------------|--------|---------------|----------------|--------------------|
| Load        | 200ps       | 100 ps        | 200ps  | 200ps         | 100 ps         | 800ps              |
| Store       | 200ps       | 100 ps        | 200ps  | 200ps         |                | 700ps              |
| Arithmetic  | 200ps       | 100 ps        | 200ps  |               | 100 ps         | 600ps              |
| conditional | 200ps       | 100 ps        | 200ps  |               |                | 500ps <sup>3</sup> |

# Pipeline Performance



# Pipeline Speedup

- If all stages are balanced

- i.e., all take the same time

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$$

- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

# Pipelining and ISA Design

Risc-V ISA designed for pipelining

- All instructions are 32-bits
  - c.f. x86: 1- to 17-byte instructions
- Few and regular instruction formats
  - Can decode and read registers in one step
- Load/store addressing
  - Can calculate address in 3<sup>rd</sup> stage, access memory in 4<sup>th</sup> stage
- Alignment of memory operands
  - Memory access takes only one cycle

# Hazards

Situations that prevent starting the next instruction in the next cycle

- Structure hazards

- A required resource is busy

- Data hazard

- Need to wait for previous instruction to complete its data read/write

- Control hazard

- Deciding on control action depends on previous instruction

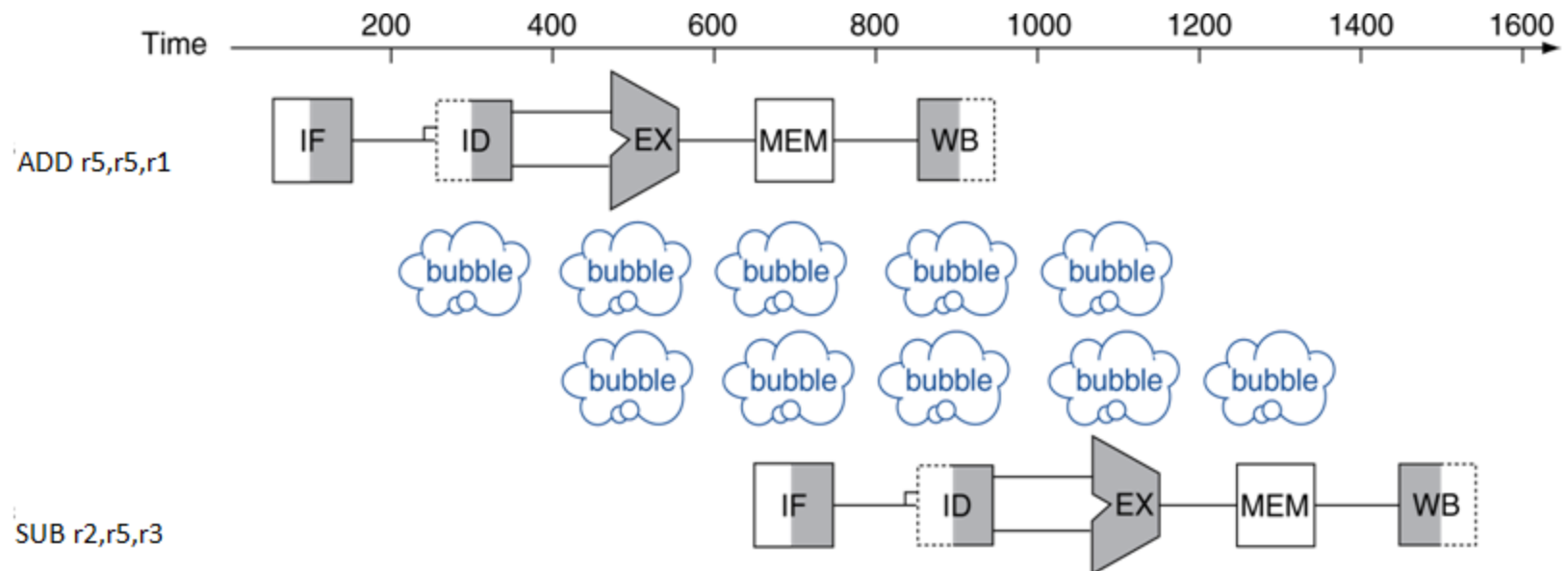
# Structure Hazards

- Conflict for use of a resource
- In MIPS pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches



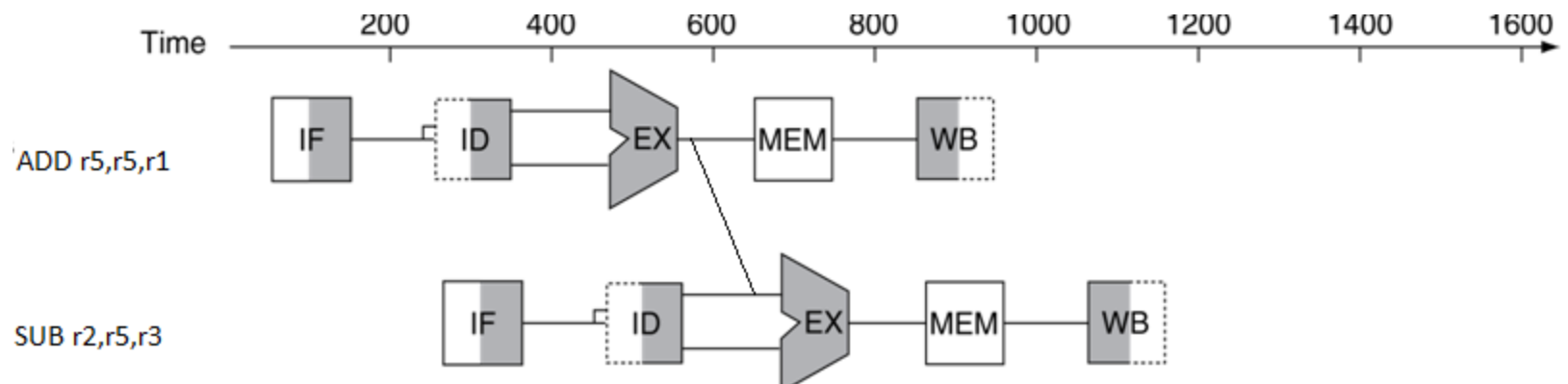
# Data Hazards

- An instruction depends on completion of data access by a previous instruction
  - ADD **r5**, r5, r1
  - SUB r2, **r5**, r3



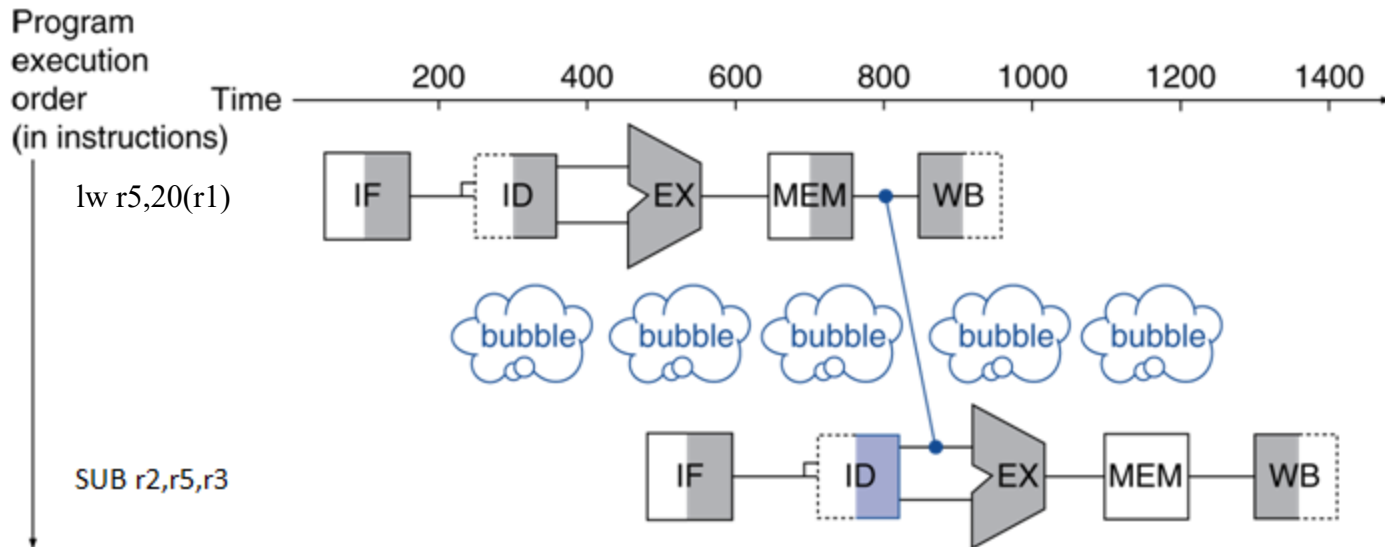
# Solution to Data Hazards: Forwarding (aka Bypassing)

- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Requires extra connections in the datapath



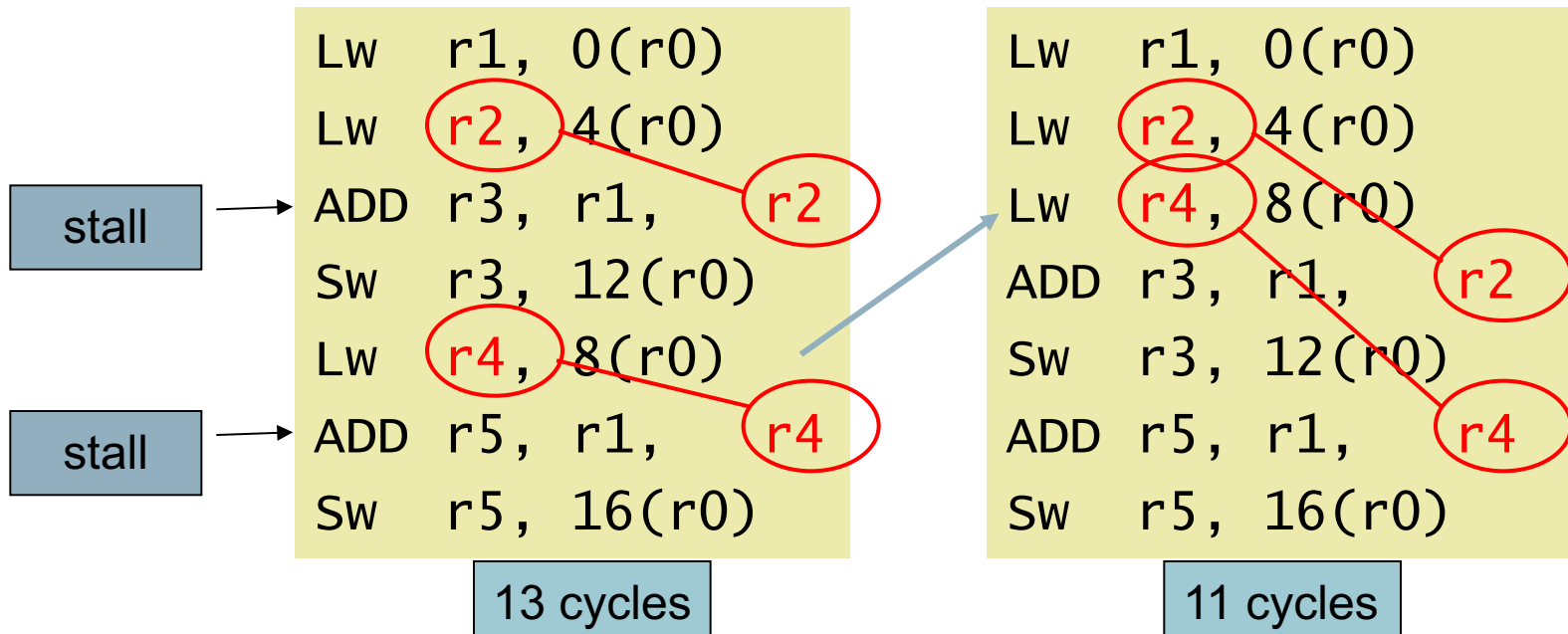
# Load-Use Data Hazard

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

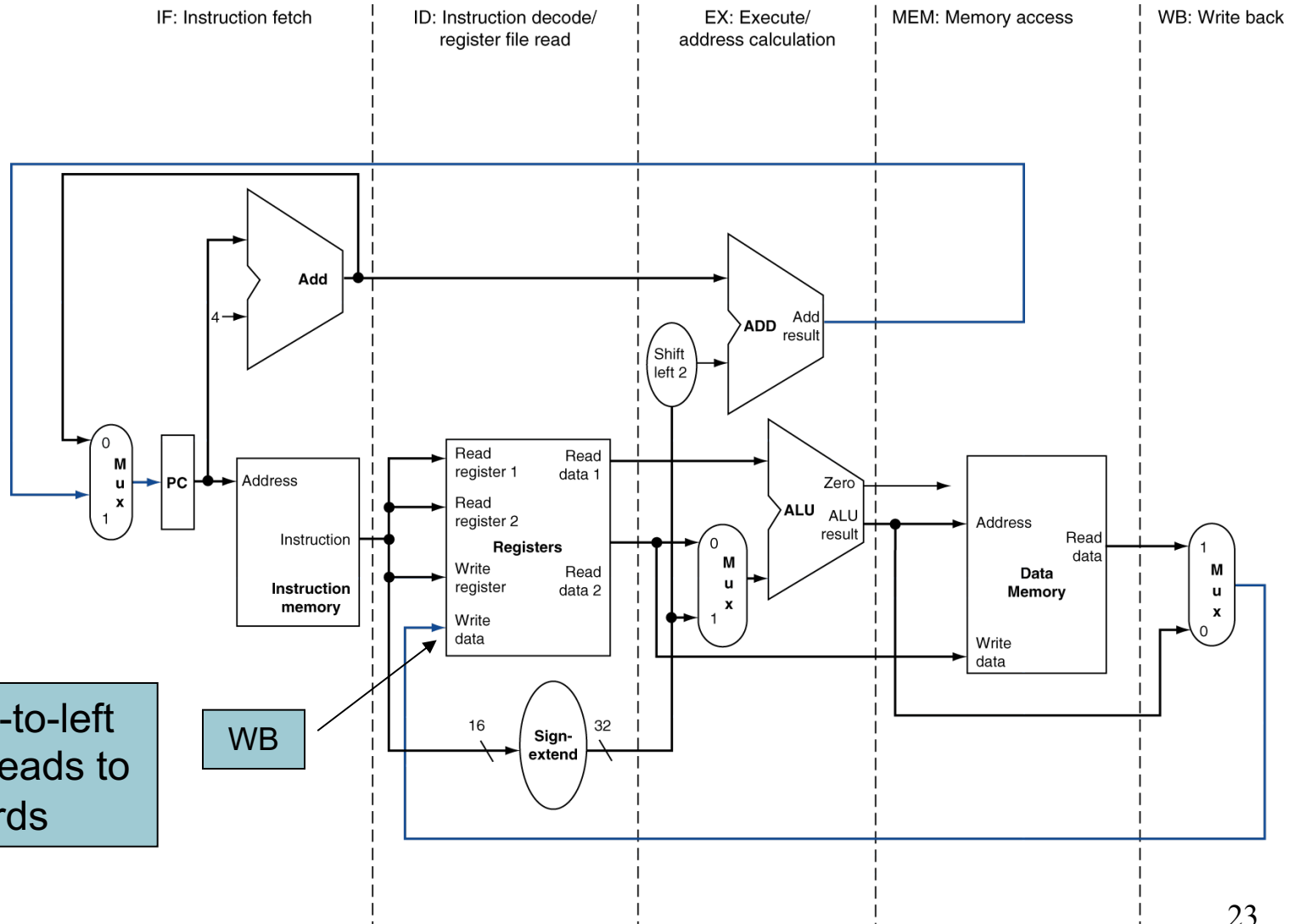


# Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for  $A = B + E$ ;  $C = B + F$ ;

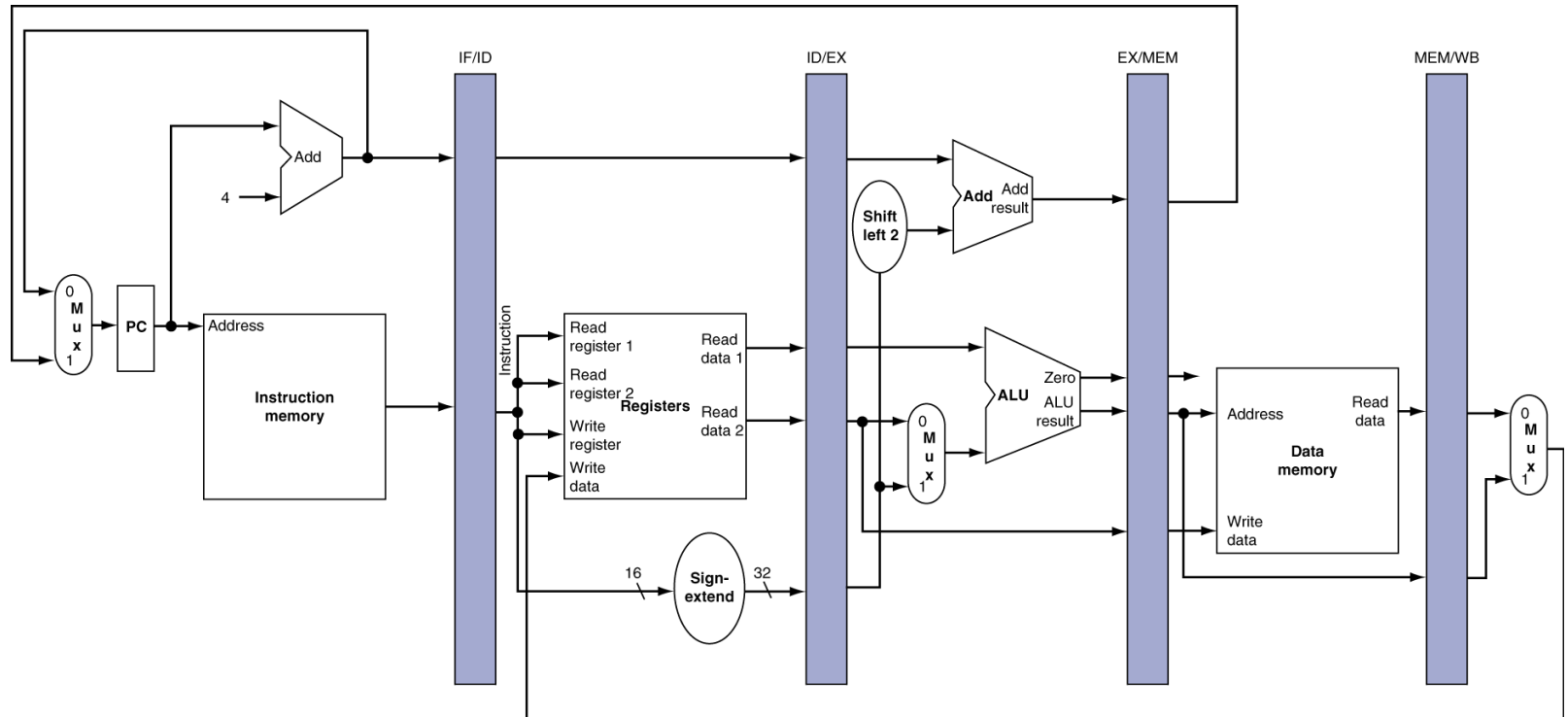


# RISC Pipelined Datapath

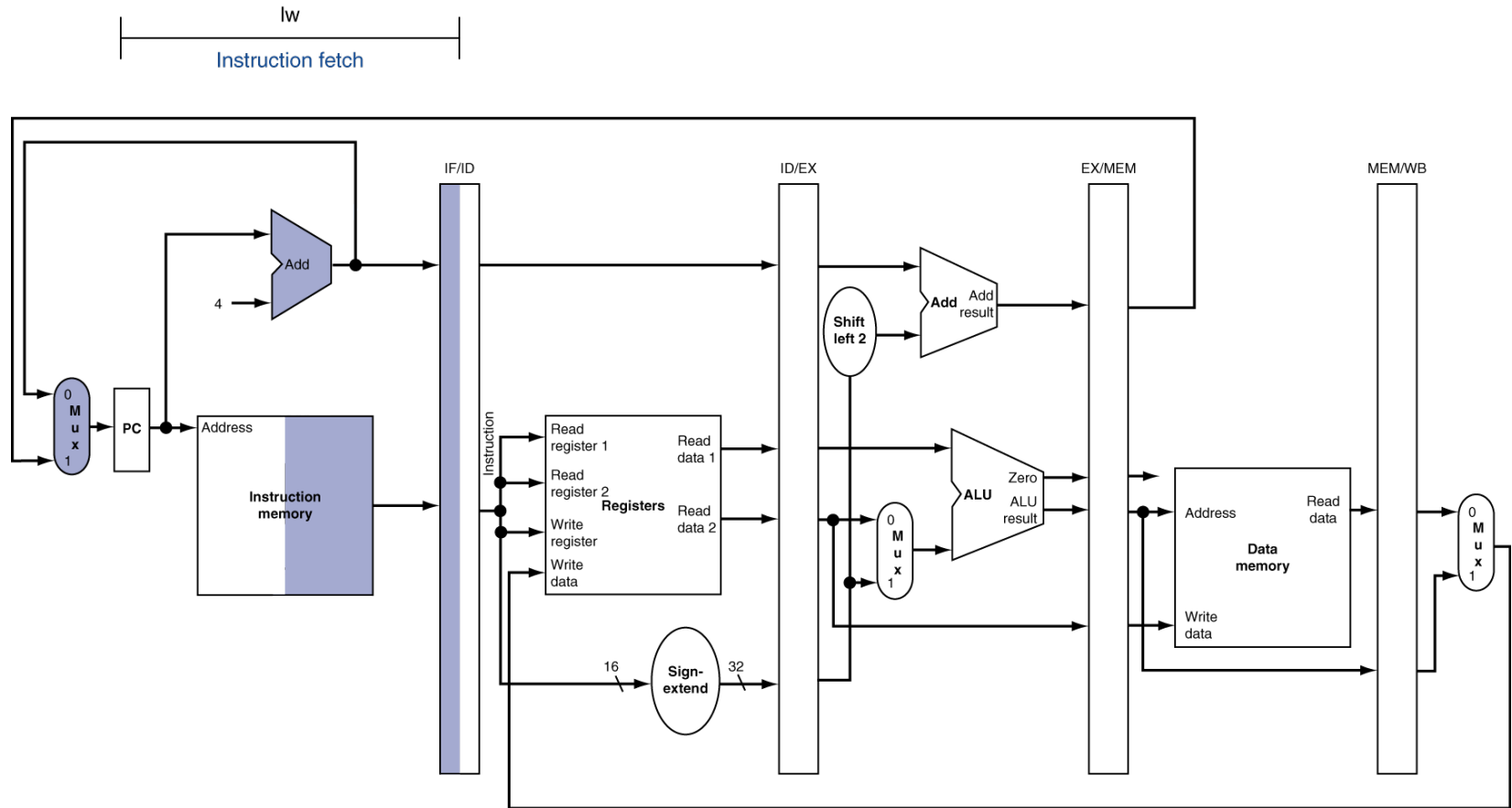


# Pipeline registers

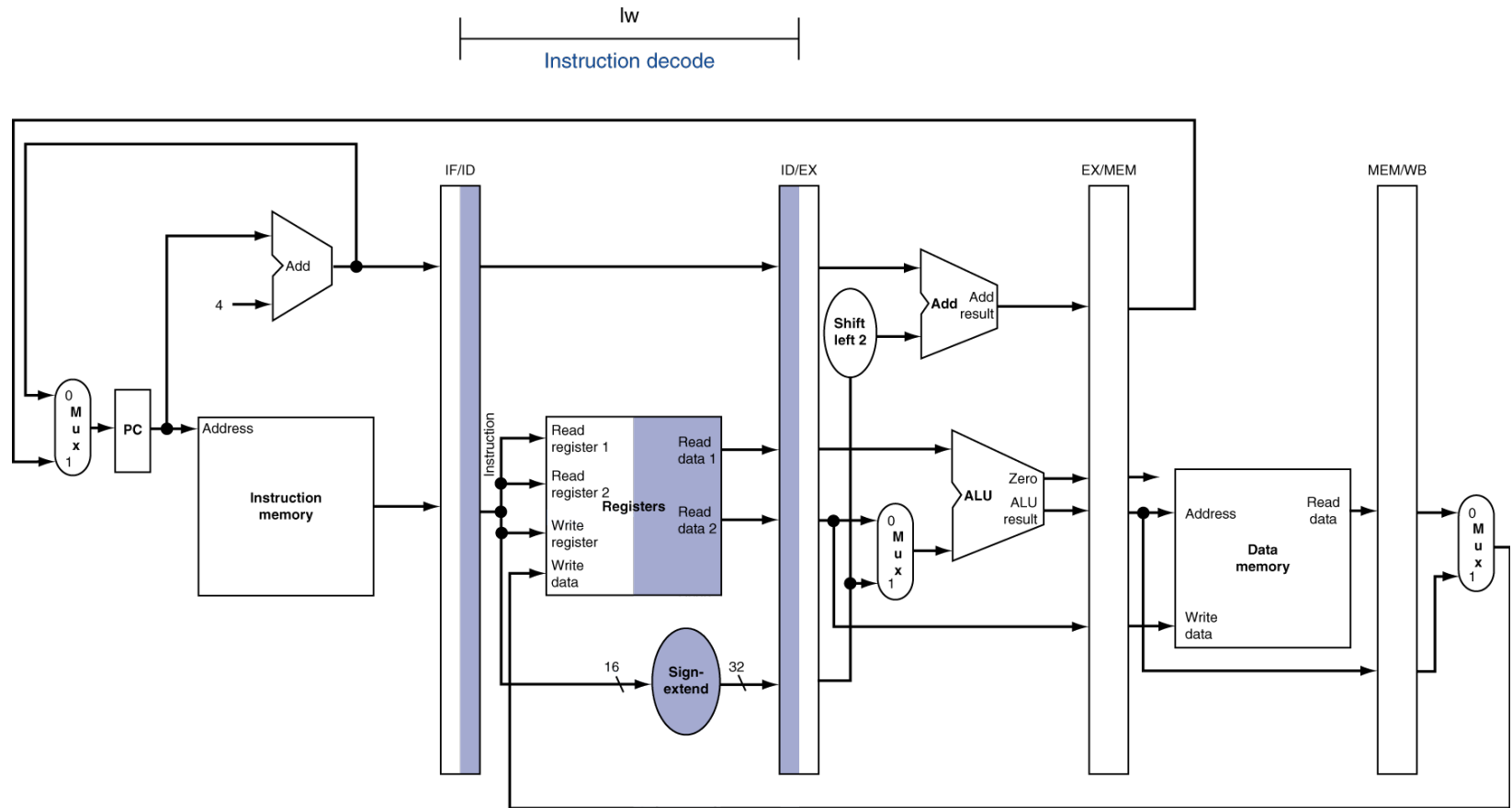
- Need registers between stages
  - To hold information produced in previous cycle



# IF for Load, Store, ...

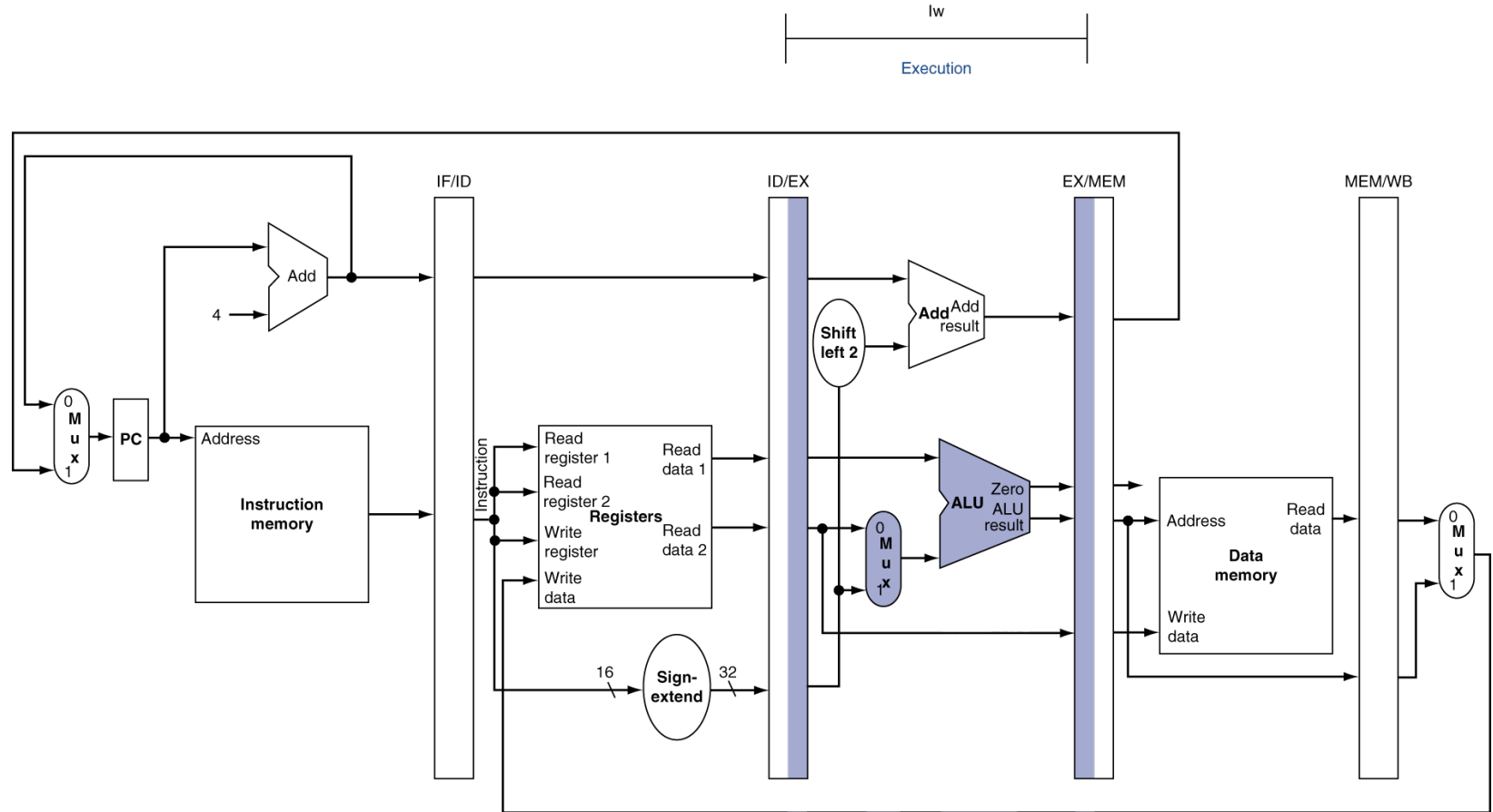


# ID for Load, Store, ...

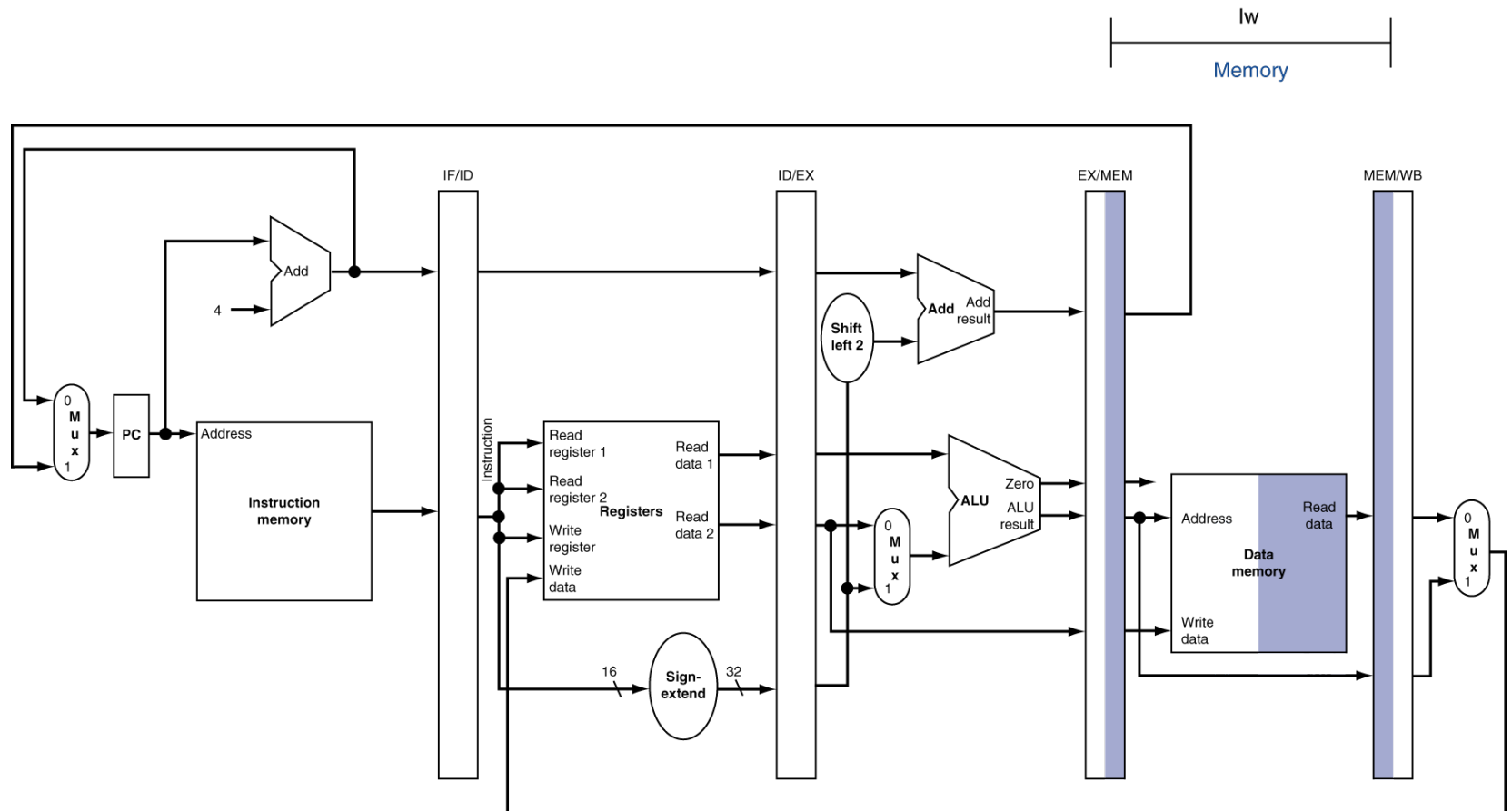




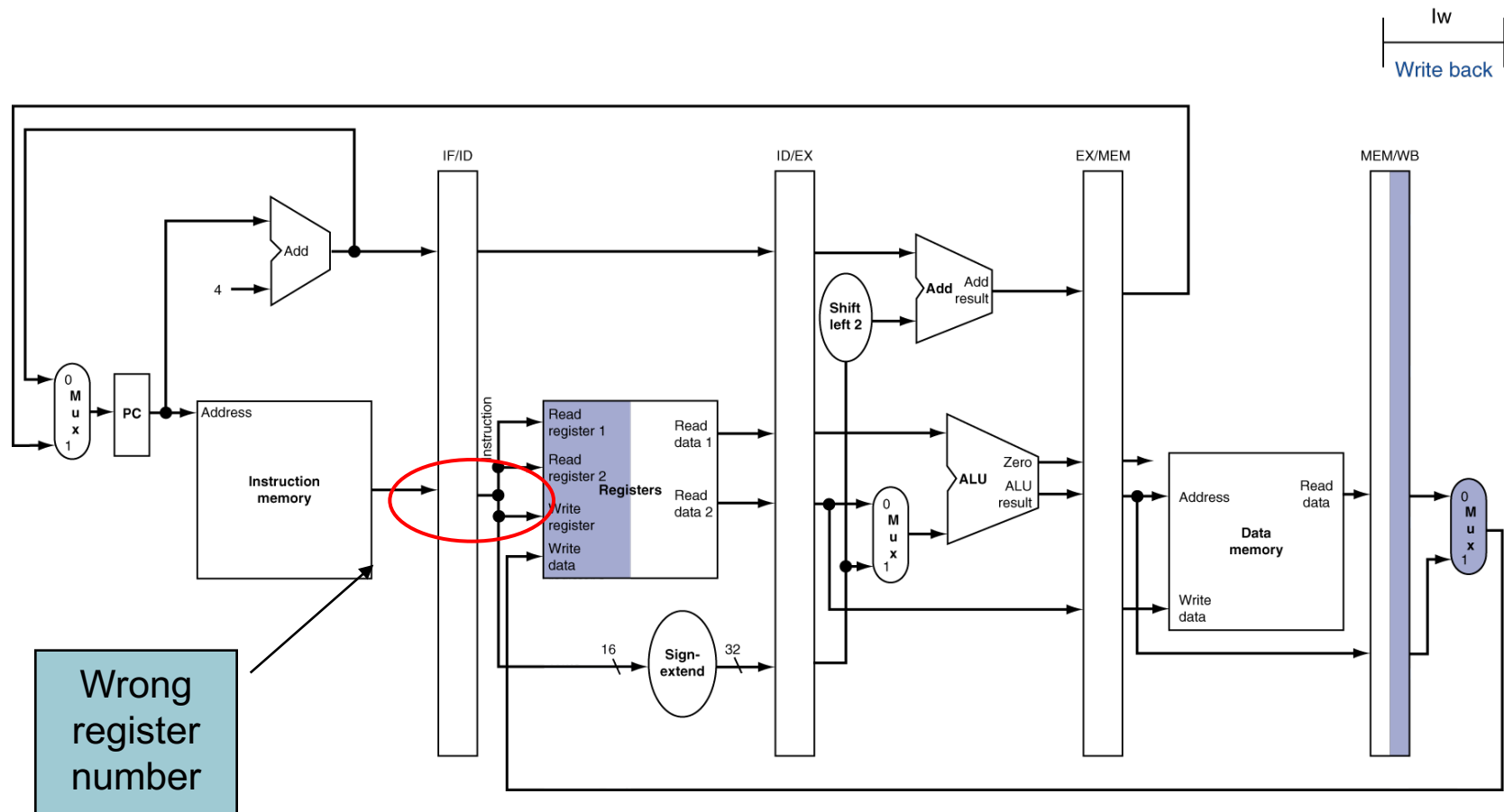
# EX for Load



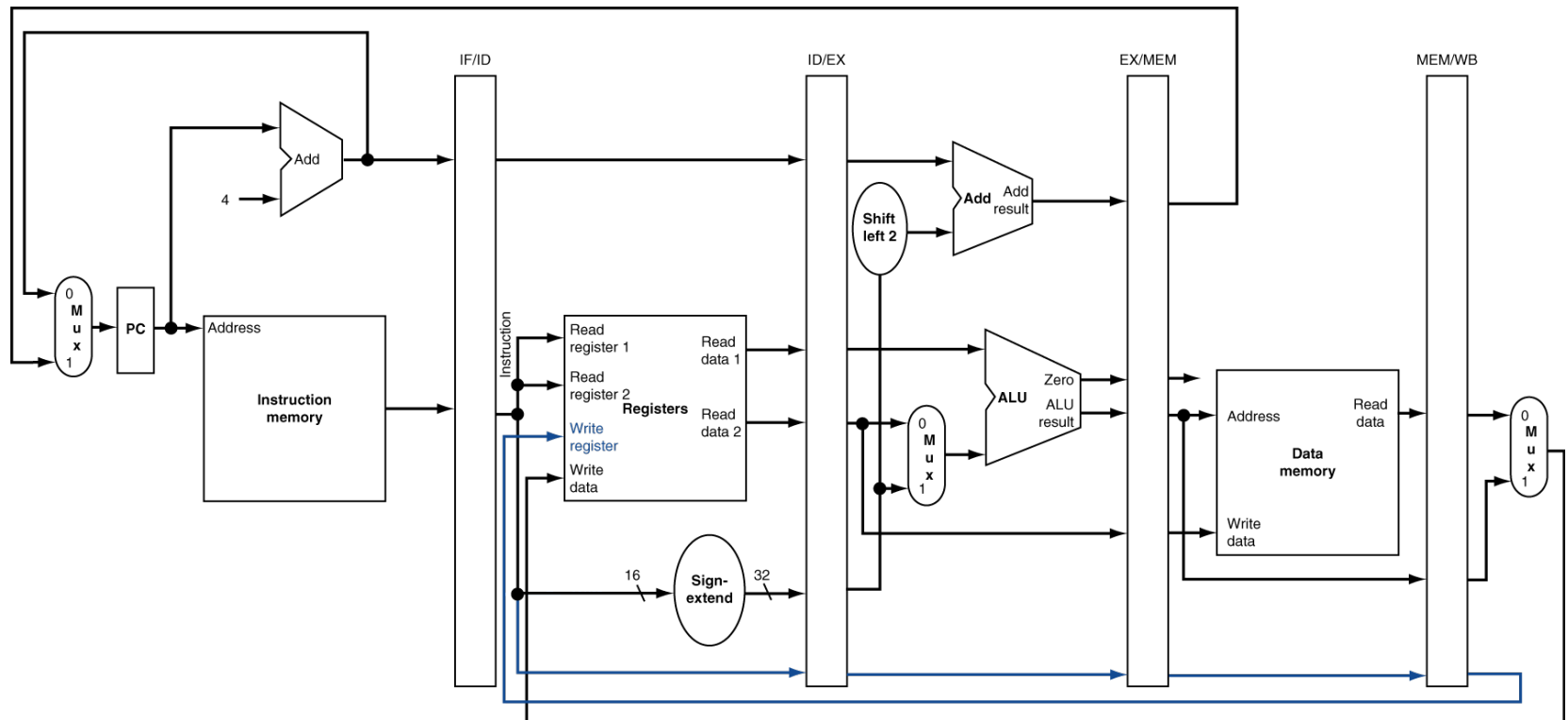
# MEM for Load



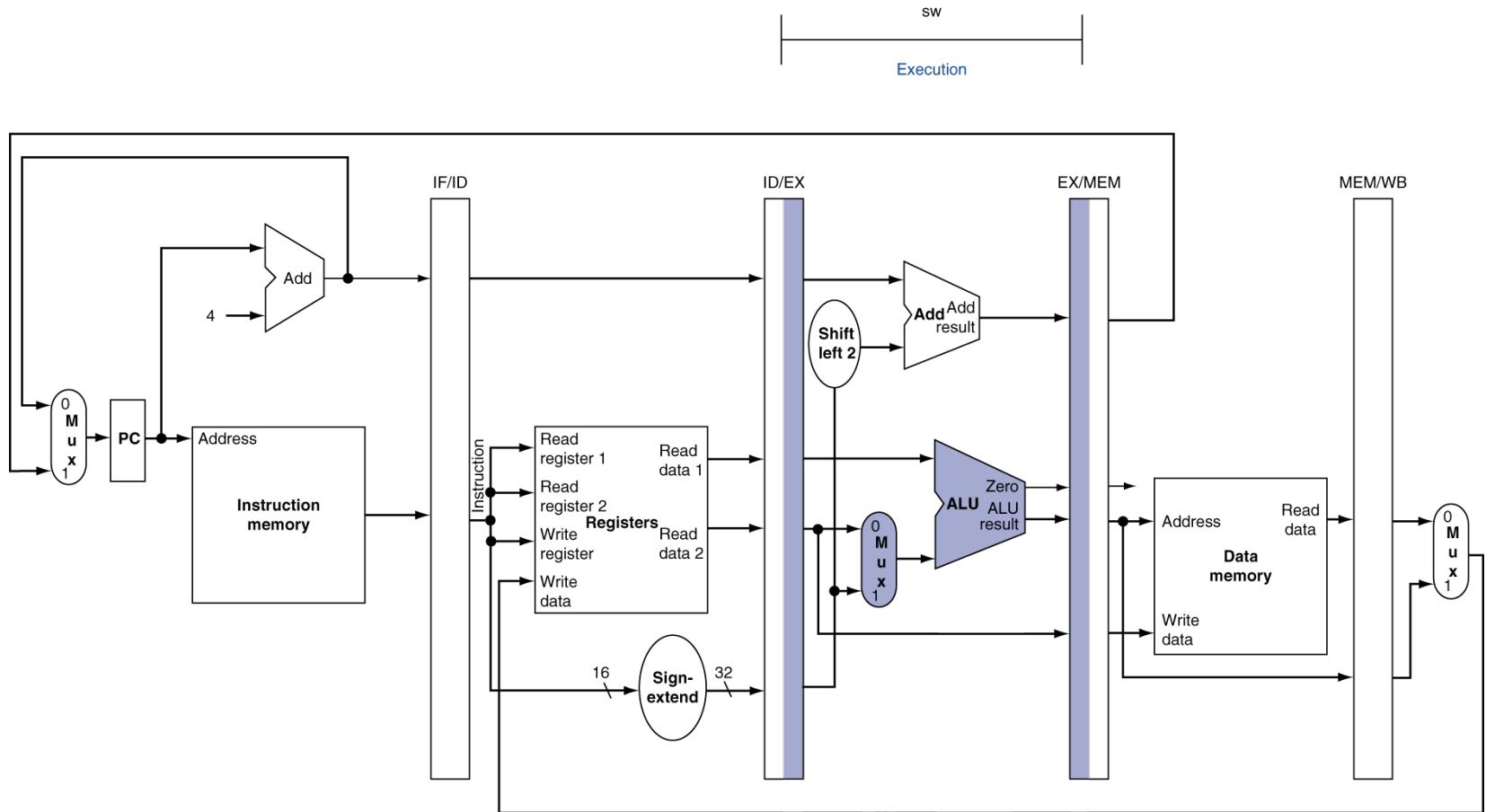
# WB for Load



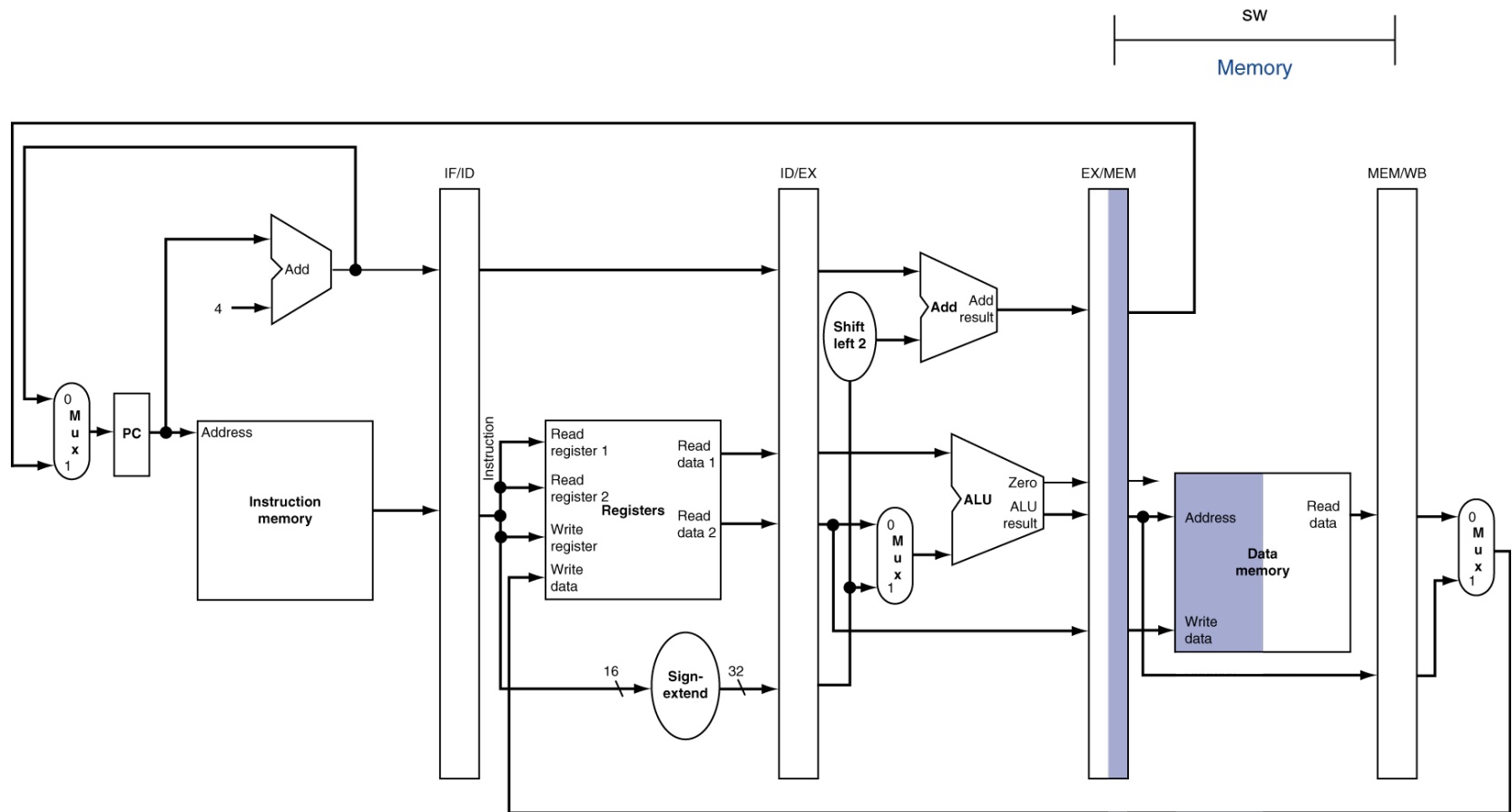
# Corrected Datapath for Load



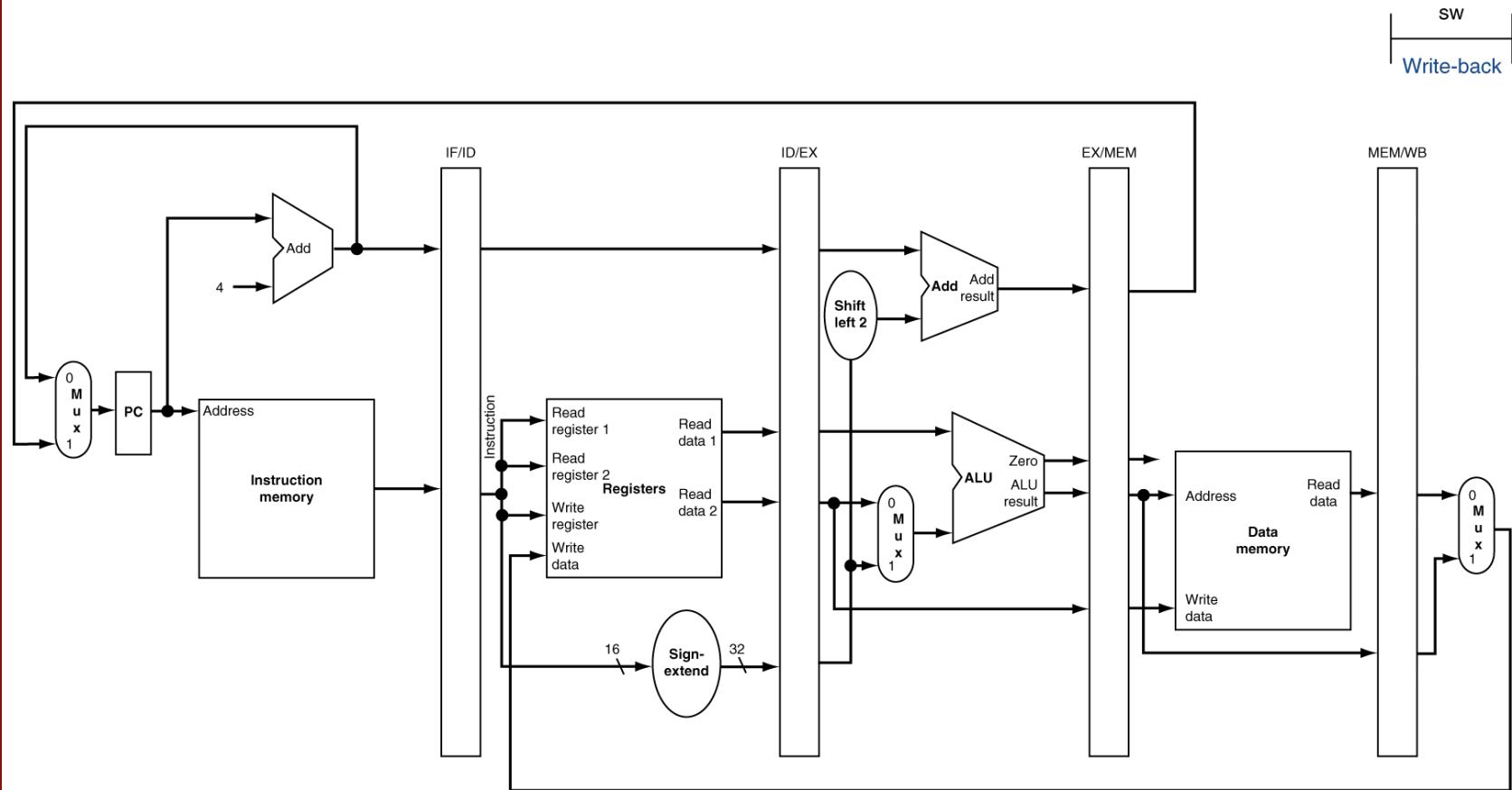
# EX for Store



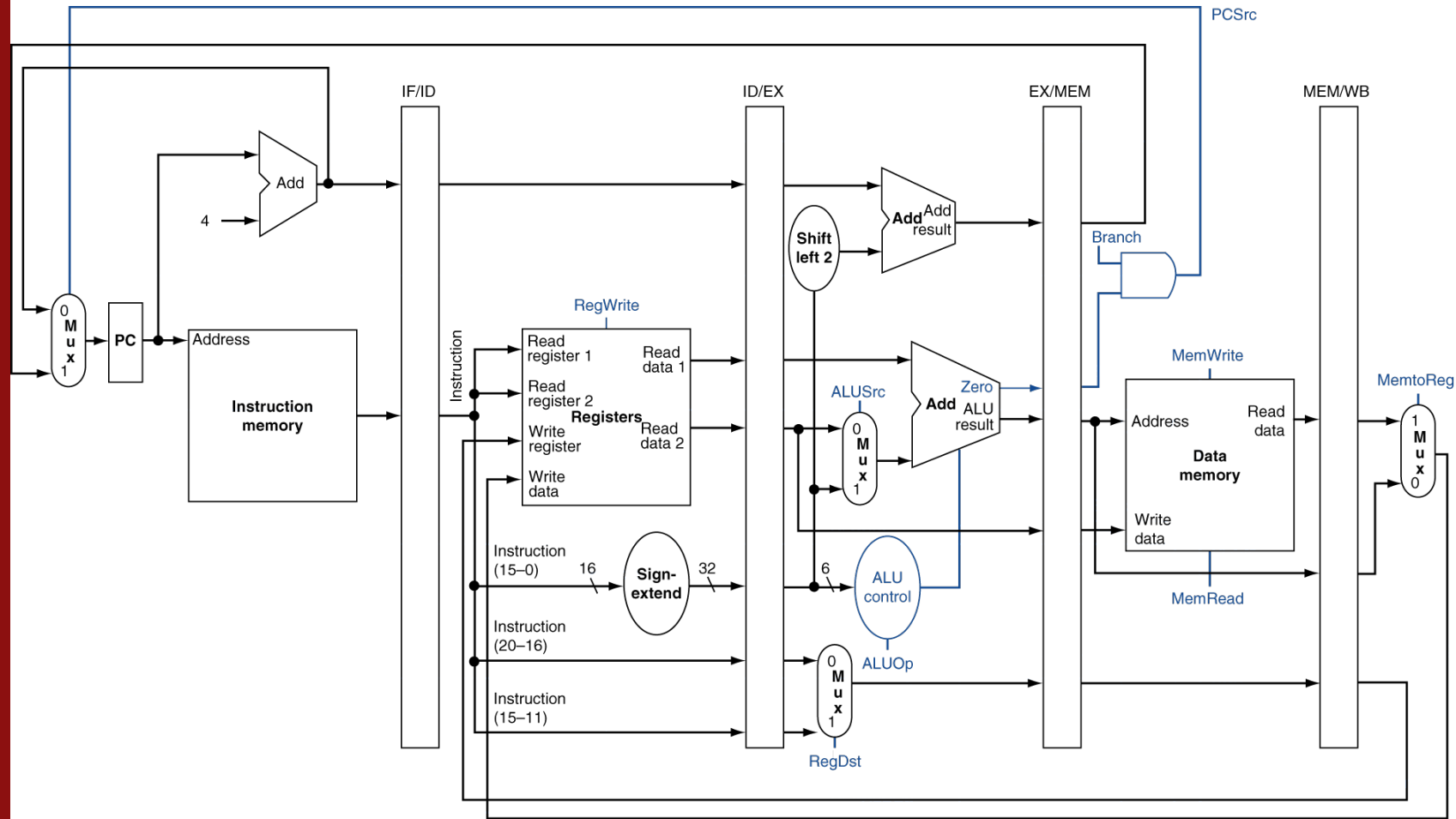
# MEM for Store



# WB for Store



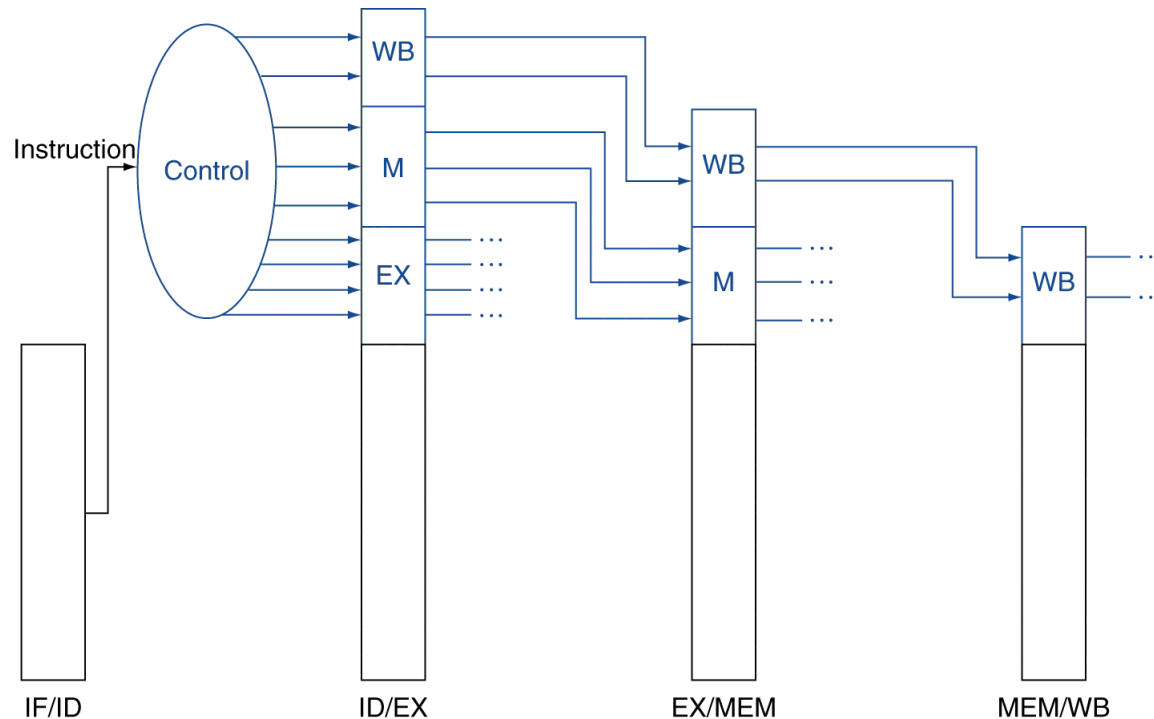
# Pipelined Control (Simplified)



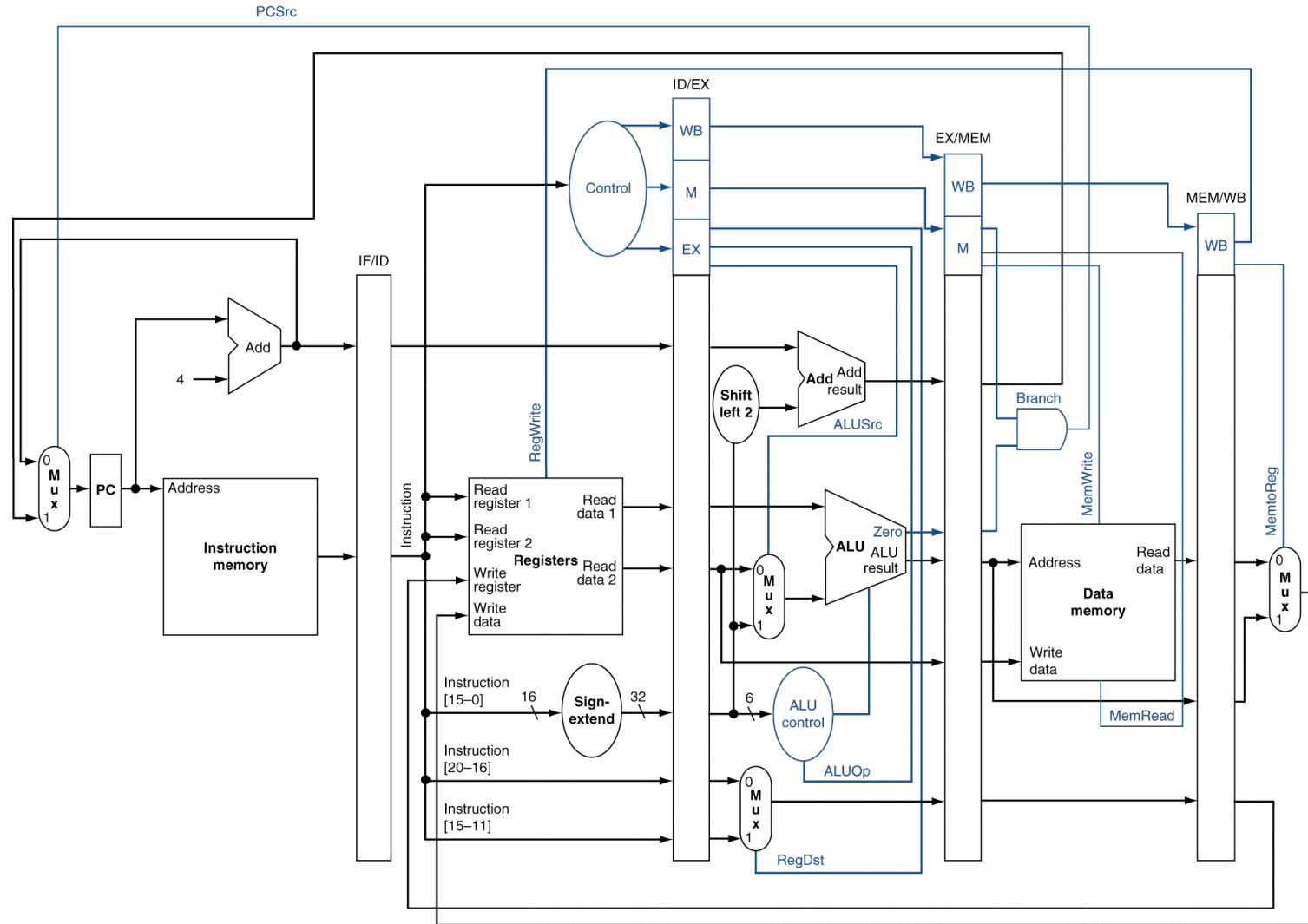


# Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



# Pipelined Control



# Data Hazards in ALU Instructions

- Consider this sequence:

```
sub  r2, r1, r3
and  r12, r2, r5
or   r13, r6, r2
add  r14, r2, r2
sw   r15, 100(r2)
```

- We can resolve hazards with forwarding
  - How do we detect when to forward?

# Detecting the Need to Forward

- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs = register number for Rs sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs, ID/EX.RegisterRt
- Data hazards when
  - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
  - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRt
  - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
  - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

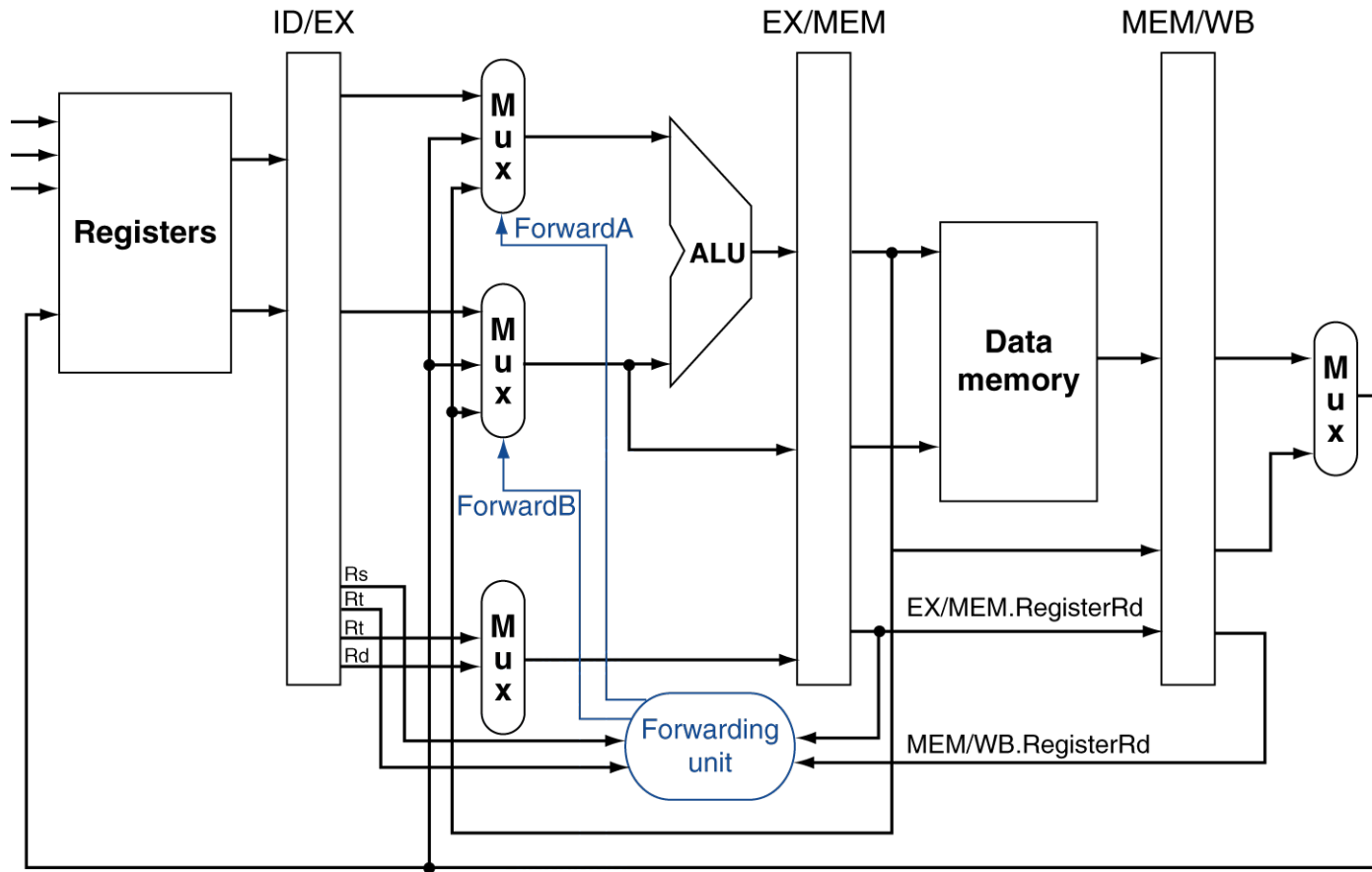
Fwd from  
EX/MEM  
pipeline reg

Fwd from  
MEM/WB  
pipeline reg

# Detecting the Need to Forward

- Some inst do not write register and will make forwarding unnecessary. But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not \$zero
  - EX/MEM.RegisterRd  $\neq 0$ ,  
MEM/WB.RegisterRd  $\neq 0$

# Forwarding Paths



b. With forwarding

# Forwarding Conditions

- EX hazard

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd == ID/EX.RegisterRs))

ForwardA = 10 //EX/MEM forwrded from ALU

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd == ID/EX.RegisterRt))

ForwardB = 10

- MEM hazard

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01 //MEM/WB forward from data memory

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01

# Double Data Hazard

- Consider the sequence:

add r1, r1, r2

add r1, r1, r3

add r1, r1, r4

- Both hazards occur

- Want to use the most recent

- Revise MEM hazard condition

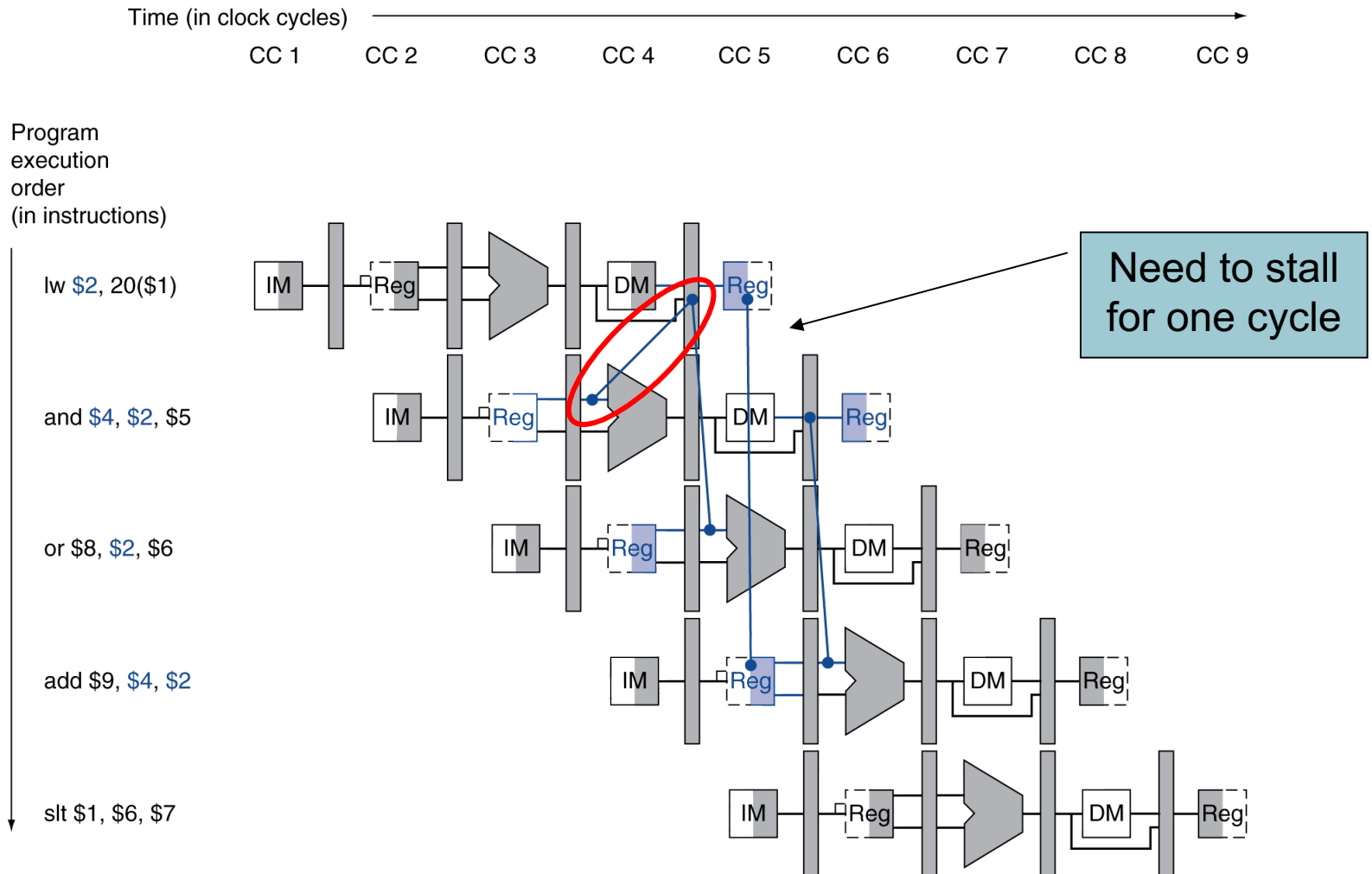
- Only fwd if EX hazard condition isn't true



# Revised Forwarding Condition

- The result is forwarded from the MEM stage because is the most recent result.
- MEM hazard
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))  
ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd  $\neq$  0)  
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd  $\neq$  0)  
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))  
ForwardB = 01

# Load-Use Data Hazard



# Load-Use Hazard Detection

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Load-use hazard when

If(ID/EX.MemRead and //checks if inst. Is lw

//check id dest. Register in EX stage matches //either source of the inst on the ID stage

((ID/EX.RegisterRt = IF/ID.RegisterRs) or  
(ID/EX.RegisterRt = IF/ID.RegisterRt)))

stall

NOTE for a load inst. We use rt register as destination

# Hazard Detection Verilog

```
Module hd(IDEX_MemRead, IDEX_rt, IFID_rs, IFID_rt, stall);  
Input IDEX_MemRead;  
Input [4:0] IDEX_rt;  
Input [31:0] IFID_rs, IFID_rt;  
output stall;  
    assign stall=(IDEX_MemRead &  
        ((IDEX_rt = IFID_rs) or  
        (IDEX_rt = IFID_rt)))  
endmodule
```

# How to Stall the Pipeline

- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for  $1w$ 
    - Can subsequently forward to EX stage

# Datapath with Hazard Detection

